

CINEMARESERVE

Cinema Booking & Reservation Management System

PROJECT REPORT

Submitted for
VITyarthi “Build Your Own Project” evaluation

Technology

Java 21 • JDBC • SQLite • JUnit 5

Author: Prince Gupta
Registration Number: 24BCY10022
September 2026

Contents

1. Introduction
 2. Problem Statement & Scope Note
 3. Functional Requirements
 4. Non-Functional Requirements
 5. System Architecture
 6. Design Decisions & Rationale
 7. Implementation Details
 8. Reservation Algorithm
 9. Scheduling Conflict Algorithm
 10. Testing Approach & Results
 11. Screenshots & Demonstration Evidence
 12. Challenges Faced & Learnings
 13. Limitations & Future Enhancements
 14. Conclusion
- Appendix A — Technology & Runtime Summary
- Appendix B — Viva / Demonstration Checklist

Report construction note

This report consolidates the supplied project-report source files into one continuous academic document. Where a supplied file points to another repository file that was not provided, the missing material is explicitly identified rather than reconstructed.

1. Introduction

CinemaReserve is a backend cinema reservation system implemented as a console (CLI) application in plain Java. It supports customer activities such as browsing movies, checking showtimes, and booking specific seats, while administrators can manage movies, theatres, screens, showtimes, and operational reports.

The central engineering focus is the reservation engine rather than CRUD alone. The system is designed to prevent a seat from being sold twice, prevent overlapping shows on the same screen, and enforce a defined cancellation policy through service-layer business rules.

Core engineering focus

The supplied introduction explicitly frames correctness under real reservation and scheduling conditions as the project's main technical emphasis, rather than treating the application as a simple CRUD exercise.

2. Problem Statement & Scope Note

The supplied Problem Statement file refers to statement.md in the repository root as the source for the full problem statement, scope, target users, and expected outcome. That repository-root file was not included among the supplied report sections.

Source boundary

To keep the report faithful to the provided material, this section does not invent or paraphrase the missing statement.md content. The functional requirements below provide the detailed scope that is present in the supplied files.

3. Functional Requirements

3.1 Customer-facing functions

ID	Requirement
FR-01	Register a new account with a unique username and email.
FR-02	Log in with username/password; inactive accounts are rejected.
FR-03	Browse and search movies by title using partial matching; genre and language filters are supported at the service layer.
FR-04	View showtimes for a selected movie and date.
FR-05	View a showtime seat map with AVAILABLE, HELD, and BOOKED states.
FR-06	Hold one or more specific seats for a showtime.
FR-07	Confirm a held reservation before the hold expires.
FR-08	View the user's reservations with status and seats.
FR-09	Cancel an eligible upcoming reservation.

3.2 Admin-facing functions

ID	Requirement
FR-10	Create, update, and deactivate movies. Deactivation is soft so historical reservations remain intact and reportable.
FR-11	Create theatres, screens with fixed capacity, and seats; support bulk seat-grid generation.
FR-12	Create showtimes while rejecting overlap with an existing SCHEDULED showtime on the same screen.
FR-13	List all upcoming showtimes.
FR-14	View reservation summary, revenue by movie, and screen occupancy percentage.

3.3 Cross-cutting requirements

ID	Requirement
FR-15	Every write operation runs in a database transaction; multi-row writes roll back on failure.
FR-16	Business-rule violations use specific named exceptions, which the CLI maps to clear single-line messages instead of raw stack traces.

4. Non-Functional Requirements

Area	Implementation / Evidence
Reliability / concurrency	Seat holds and confirmations use BEGIN IMMEDIATE write transactions; UNIQUE(showtime_id, seat_id) provides the database-enforced double-booking guarantee. The concurrency test uses 12 threads across 5 repetitions.
Security	BCrypt password hashing with work factor 12, service-layer role authorization for admin methods, and parameterized SQL throughout.
Maintainability	Strict CLI → Service → DAO → DB layering, specific exception types, and small classes without framework-heavy behavior.
Usability	Centralized CLI error handling in Main.act() converts business exceptions to one-line human-readable messages.
Performance	Reports use SQL GROUP BY/SUM/COUNT aggregation; indexes support showtime, user, and reservation lookups.
Data integrity	Foreign keys are enabled; CHECK constraints protect statuses, durations, prices and time ordering; UNIQUE constraints protect usernames, emails, booking references and seat/showtime pairs.
Error handling	Service methods use specific exceptions and explicit rollback paths for write transactions.
Resource efficiency	One JDBC connection is used per unit of work and closed after the operation; SQLite WAL mode supports concurrent reads.

5. System Architecture

CinemaReserve follows a strict four-layer architecture. The separation keeps presentation, business rules, data access, and persistence responsibilities distinct.

```

CLI
(Main.java, ApplicationContext, SeedData)
    |
    v
Service Layer
(AuthService, MovieService, VenueService,
 SchedulingService, ReservationService,
 ReportingService)
    |
    v
DAO Layer
(UserDao, MovieDao, GenreDao, TheatreDao,
 ScreenDao, SeatDao, ShowtimeDao,
 ReservationDao)
    |
    v
SQLite Database
(data/cinemareserve.db)

```

5.1 Layer responsibilities

CLI layer — Pure presentation. It reads input, calls one service method per action, and prints results or friendly errors. It contains no business logic or SQL.

Service layer — Owns validation, authorization, transaction boundaries, and the reservation and scheduling algorithms. Services depend on DAOs and domain classes, not on the CLI.

DAO layer — Provides one data-access class per aggregate/table. Queries use PreparedStatement parameters, and DAOs map result sets to domain objects without embedding business rules.

Database layer — Uses one SQLite file. UNIQUE, CHECK, and FOREIGN KEY constraints provide a second line of defense, including the seat double-booking guarantee.

5.2 Transaction boundaries

A write transaction corresponds to a single service method. The supplied architecture identifies holdSeats, confirmReservation, cancelReservation, and createShowtime as examples. Each opens a connection, begins the write transaction, performs reads/writes, and commits once or rolls back on exception.

5.3 Security flow

1. AuthService.login looks up the user, checks active status, and verifies the password with PasswordHasher.matches using BCrypt.
2. The CLI retains the authenticated User object as currentUser for the session.
3. Admin-only service methods receive the acting User and check authorization in the service layer before performing the operation, preventing the CLI from becoming the only security boundary.

5.4 Reservation flow

```
Customer → holdSeats → HELD reservation
Customer → confirmReservation → CONFIRMED reservation
Customer → cancelReservation → eligible cancellation → released seats
```

Architecture reference

The supplied architecture file also identifies repository Mermaid diagrams for the system, component, and hold/confirm sequence views. Those source diagrams were not supplied as image assets, so this report uses a clean text reconstruction of the documented four-layer structure.

6. Design Decisions & Rationale

6.1 Plain JDBC + SQLite

The build/verification environment had no Maven Central access. The project therefore used locally available JARs rather than delivering untested Spring Boot code. The resulting stack requires no external services at runtime.

6.2 Master seat vs. showtime-specific state

A Seat represents a physical seat independently of a showtime. Occupancy is stored through reserved_seats and reservations, allowing the same physical seat to have different availability across different shows.

6.3 UNIQUE(showtime_id, seat_id)

The unique database constraint is the actual double-booking guarantee. The initial availability read is only for a fast, specific user-facing error; correctness is enforced by the insert constraint.

6.4 BEGIN IMMEDIATE

The first concurrency test exposed SQLITE_BUSY failures caused by SQLite deferred-transaction lock upgrades. BEGIN IMMEDIATE acquires the write lock up front so concurrent writers queue instead of racing during lock upgrade.

6.5 Soft deletion for movies

Movie status is ACTIVE/INACTIVE rather than deleting the row, preserving historical reservations and reports.

6.6 Custom exception hierarchy

Distinct exception types represent validation, authorization, seat availability, scheduling, reservation state, not-found, and authentication failures. The CLI can therefore map categories directly to user-friendly messages.

6.7 No ORM

With plain JDBC, SQL remains visible in the DAO that issues it. For the project's eight-table scale, the added mapping code was considered acceptable for transparency and viva/debugging clarity.

7. Implementation Details

7.1 Package layout

com.cinemareserve.domain	Plain domain objects + enums
com.cinemareserve.dao	JDBC data access
com.cinemareserve.service	Business rules, transactions, authorization
com.cinemareserve.exception	Custom business exceptions
com.cinemareserve.security	BCrypt password hashing
com.cinemareserve.db	Connection, schema bootstrap, transactions
com.cinemareserve.util	Validator, BookingReferenceGenerator
com.cinemareserve.cli	Main, AppContext, SeedData

7.2 Notable implementation points

- AppContext is the composition root and manually wires DAOs into services without a dependency-injection framework.
- Each service-layer unit of work opens and closes its JDBC connection using try-with-resources; connections are not retained across user interactions.
- Database.getConnection() bootstraps schema.sql once per process using IF NOT EXISTS.
- ReservationService.priceFor adds a fixed surcharge for PREMIUM and RECLINER seats over the showtime base price.
- BookingReferenceGenerator creates short human-readable references such as CR-7F3K9Q and avoids visually ambiguous characters such as 0/O and 1/I.

7.3 Runtime environment

Component	Known-good environment
Java	Java 21
Database	SQLite via sqlite-jdbc 3.44.x
Password hashing	jBCrypt 0.4
Logging support	slf4j-api + slf4j-simple 1.7.x
Testing	JUnit Platform Console Standalone
Build style	Local JARs with javac/java; Maven not required

8. Reservation Algorithm

8.1 Race condition

If two users attempt to hold the same seat within milliseconds, both can observe the seat as free in a naive check. If both then insert a booking, the application can create two reservations for one seat.

8.2 Chosen locking strategy

4. reserved_seats has a UNIQUE(showtime_id, seat_id) constraint.
5. holdSeats first reads blocked seats only to produce a fast and specific message such as “seat A2 already held or booked.”
6. The subsequent INSERT is the correctness mechanism: the unique index turns the insert into an atomic compare-and-insert.
7. If SQLite raises SQLITE_CONSTRAINT_UNIQUE, ReservationService converts it into SeatUnavailableException.
8. Write transactions use BEGIN IMMEDIATE so concurrent writers queue for the write lock.

8.3 Concurrent execution trace

Step	What happens
1	Both transactions begin; BEGIN IMMEDIATE serializes access to the write path.
2	The first transaction inserts the reserved seat and commits.
3	The second transaction reaches the insert and finds the same (showtime_id, seat_id).
4	The second insert fails with a unique-constraint violation; the transaction rolls back.
5	The first caller receives a HELD reservation; the second receives SeatUnavailableException. No duplicate active row exists.

Automated proof

ReservationConcurrencyTest starts 12 threads against the same seat/showtime, expects exactly 1 success and 11 SeatUnavailableException results, and repeats the test 5 times. The supplied report states that all 5 repetitions pass and that the database contains exactly one reservation row afterward.

9. Scheduling Conflict Algorithm

9.1 Overlap condition

For half-open intervals [startA, endA) and [startB, endB), overlap occurs if and only if:

```
startA < endB   AND   startB < endA
```

The strict comparisons also make back-to-back shows valid: if endA equals startB, the intervals do not overlap.

9.2 Implementation

```
SELECT ... FROM showtimes
WHERE screen_id = ?
  AND status = 'SCHEDULED'
  AND start_time < ?
  AND ? < end_time
```

`SchedulingService.createShowtime` performs the overlap query inside a `BEGIN IMMEDIATE` transaction. Before the overlap check, the service also verifies that the movie and screen are active, that start is strictly before end, and that the acting user is an administrator.

9.3 Test coverage

- Valid showtime creation succeeds.
- An overlapping showtime is rejected with `ScheduleConflictException`.
- Back-to-back non-overlapping showtimes are accepted.
- Invalid time windows are rejected.
- Non-admin users are rejected.

10. Testing Approach & Results

All tests use JUnit 5 and run through junit-platform-console-standalone.jar, so Maven is not required. Each test receives a fresh isolated SQLite database created through TestDb.freshDatabase(), preventing test state from leaking between cases.

10.1 Test coverage

Test class	Coverage
AuthServiceTest	Hashed passwords; duplicate username/email rejection; password validation; successful login; wrong password and unknown-user rejection.
SchedulingServiceTest	Valid creation; overlap rejection; back-to-back boundary case; invalid time window; non-admin rejection.
ReservationServiceTest	Seat holds; unavailable seats; multi-seat holds; duplicate/unknown seats; confirmation; ownership; cancellation rules; release after cancellation; booking after showtime start.
ReservationConcurrencyTest	12 concurrent users race for one seat; exactly 1 succeeds and 11 fail; repeated 5 times. Separate test verifies distinct concurrent seats do not interfere.
ReportingServiceTest	Confirmed-ticket summary and revenue; revenue by movie; screen occupancy; admin-only reporting access.

10.2 Result

Verified result in supplied report

36 tests, 36 passing, 0 failing, 0 skipped. The supplied testing file identifies test_run.txt as the captured console output produced by scripts/test.sh.

10.3 Testing boundary

- The CLI layer is manually exercised because it contains orchestration and printing rather than business logic.
- MovieService and VenueService CRUD paths are exercised indirectly through TestFixtures instead of having dedicated test classes.
- These are documented coverage boundaries rather than hidden assumptions.

11. Screenshots & Demonstration Evidence

The supplied screenshot specification states that these must be captured from an actual run on the evaluator's/student's machine. No fabricated screenshots are inserted into this report.

#	Evidence to capture	Status
1	Application startup banner (./scripts/run.sh)	Capture during final demo
2	Registration flow	Capture during final demo
3	Successful customer login as alice	Capture during final demo
4	Browse/search movies	Capture during final demo
5	View showtimes for a movie and date	Capture during final demo
6	Seat map showing AVAILABLE / HELD / BOOKED	Capture during final demo
7	Successful seat hold with booking reference	Capture during

#	Evidence to capture	Status
		final demo
8	Confirming a held reservation	Capture during final demo
9	My reservations showing a CONFIRMED booking	Capture during final demo
10	Rejected booking attempt for an already held/booked seat	Capture during final demo
11	Cancelling an eligible reservation	Capture during final demo
12	Login as admin	Capture during final demo
13	Admin menu	Capture during final demo
14	Successful showtime creation	Capture during final demo
15	Rejected overlapping showtime creation	Capture during final demo
16	Admin reports: summary, revenue by movie, screen utilization	Capture during final demo
17	Test output showing 36/36 tests passing, including concurrency repetitions	Capture during final demo

Important

The source file explicitly says these images are not included in the repository and must be captured from an actual run. The report therefore uses an evidence checklist rather than simulated terminal screenshots.

12. Challenges Faced & Learnings

12.1 Challenges faced

9. No Maven Central access: the project pivoted to plain JDBC + SQLite and vendored dependencies as local JARs.
10. SQLITE_BUSY under concurrent load: the issue was traced to SQLite deferred-transaction lock upgrades and fixed using BEGIN IMMEDIATE.
11. SQLite JDBC generated-key support: getGeneratedKeys() was not fully supported by the driver, so SELECT last_insert_rowid() was used on the same connection after inserts.
12. Platform-specific SQLite native libraries: an unsuitable Debian-packaged JAR caused native-library problems on Windows; it was replaced with the official multi-platform Xerial JAR.
13. Naive SQL-comment stripping: splitting schema.sql directly on semicolons failed when comments contained semicolons; comments were stripped before statement splitting.

12.2 Key learnings

- A database uniqueness constraint can provide a stronger correctness guarantee than repeated application-level availability checks for a race condition.
- SQLite's single-writer model and deferred versus immediate transactions have important consequences for concurrent JDBC applications.
- Environment constraints should be discovered early and incorporated into architecture decisions rather than assumed away.

13. Limitations & Future Enhancements

The supplied source does not enumerate the actual contents of the README's Known Limitations or Future Enhancements sections; it only points to those sections. To avoid inventing material, this report preserves that boundary.

13.1 Documented limitation

Automated tests do not directly exercise the CLI layer, and MovieService/VenueService CRUD paths are covered indirectly through TestFixtures. These boundaries are explicitly documented in the supplied testing approach.

13.2 Natural extension

The service layer is UI-agnostic. The supplied conclusion identifies a REST API or web front end as a natural next step that can be added without rewriting the core service design.

14. Conclusion

CinemaReserve implements the supplied functional and non-functional requirements with a deliberately minimal and transparent technology stack. Its main engineering claims—preventing double-booking under concurrent access and rejecting overlapping showtimes on the same screen—are supported by automated, repeatable tests.

The project also records a real concurrency issue discovered during development: SQLite lock-upgrade contention. The issue was root-caused and addressed through BEGIN IMMEDIATE rather than being hidden behind a generic timeout adjustment.

Because the service layer is independent of the CLI, the project can be extended toward a REST API or web front end without requiring a rewrite of its core reservation and scheduling logic.

Appendix A — Technology & Runtime Summary

Layer / Concern	Technology / Approach
Language	Java 21
Database	SQLite
Data access	JDBC / PreparedStatement
Authentication	BCrypt password hashing
Application interface	Console / CLI
Testing	JUnit 5
Concurrency control	BEGIN IMMEDIATE + UNIQUE(showtime_id, seat_id)
Transactions	Explicit service-layer write transactions with rollback
Persistence constraints	UNIQUE, CHECK, FOREIGN KEY
Logging support	SLF4J API + simple implementation
Dependency management for supplied environment	Vendored local JARs

Appendix B — Viva / Demonstration Checklist

- Explain why the seat availability read alone cannot prevent double booking.
- Explain how UNIQUE(showtime_id, seat_id) provides the database-level guarantee.
- Explain why BEGIN IMMEDIATE was selected over SQLite's deferred transaction mode.
- Demonstrate a successful customer reservation from hold to confirmation.
- Demonstrate rejection of an already held/booked seat.
- Demonstrate scheduling rejection for an overlapping showtime.
- Explain the interval condition: startA < endB AND startB < endA.
- Show the 36/36 JUnit result.
- Explain the four-layer architecture and why business rules belong in the service layer.
- Explain why soft deletion preserves historical movie reservations.
- Mention the documented testing boundaries rather than claiming untested CLI behavior is automated.

References

- SQLite documentation — transaction and locking behavior.
- Xerial sqlite-jdbc driver documentation.
- jBCrypt documentation.
- JUnit 5 User Guide.
- Oracle Java 21 documentation.

Source files consolidated

01-cover-page through 14-conclusion, as supplied for this report. Repository-only references such as statement.md, README sections, test_run.txt, and Mermaid diagrams are retained as references where the supplied files point to them, but their unavailable contents are not fabricated.